

MCP

Connect to MCP servers

Add an MCP server to Claude Code, verify the connection, and find the configuration on disk.

The **Model Context Protocol (MCP)** lets Claude Code use tools beyond its built-in set, such as searching an issue tracker, querying a database, or controlling a web browser. These tools come from MCP servers, which run on your machine or as hosted services.

This guide walks you through connecting one MCP server end to end with the Claude Code CLI. By the end, you'll have a server connected and responding, know where its configuration lives on disk, and know how to fix the most common connection errors.

! You can also add MCP servers from other surfaces, including the desktop app, VS Code, and the web. See [Connect from other surfaces](#).

For every way to connect and configure MCP servers in Claude Code, see the **MCP reference**.

Before you begin

Make sure you have:

- **Claude Code installed** and authenticated
- A terminal open in a project directory. Any directory works, including an empty one.

Add and verify a server

The example below connects to the **Claude Code documentation MCP server**, a hosted server with full-text search over the Claude Code docs. It doesn't require authentication or any special configuration, so it works well as a first server to test the setup flow with.

The steps are the same for any server: add it, check the connection status, then use it in a session, with an optional cleanup step at the end. Some servers add a step, like a browser sign-in, shown in **Additional MCP server examples**. For more servers to connect, browse the **Anthropic Directory**.

1 Add the MCP server

Register the server with Claude Code. Run this in your terminal, not inside a `claude` session: you're configuring the server before starting a conversation.

```
claude mcp add --transport http claude-code-docs
https://code.claude.com/docs/mcp
```

The parts of the command:

- `claude mcp add` : registers a server with Claude Code.
- `--transport http` : the server is hosted at a URL rather than run as a local process.
- `claude-code-docs` : a name you make up. Calling the same server `docs` would work identically. Claude Code uses whatever name you pick to label the server's tools in Claude's output and to refer to the server in commands like `claude mcp remove`.
- `https://code.claude.com/docs/mcp` : the URL where the server is hosted.

The command prints a confirmation like `Added HTTP MCP server claude-code-docs with URL: https://code.claude.com/docs/mcp to local config`. The `local config` part means the server is registered to you, in this project: if you start Claude Code in a different project, this server isn't active there. To register a server once for all your projects, add it at user scope, covered in [Change server scope](#).

2 Check the connection status

Confirm the server appears in your server list and check its status:

```
claude mcp list
```

The server appears with a status indicator:

Status	Meaning
✓ Connected	Ready to use. This is what you should see for <code>claude-code-docs</code>
! Connected • tools fetch failed	The server connected but couldn't list its tools. Run <code>claude mcp get <name></code> for the error detail

Status	Meaning
! Needs authentication	The server is reachable but needs a browser sign-in, or a token passed with <code>--header</code> . See Connect a server that requires sign-in
X Failed to connect	Server didn't respond. See Troubleshooting
X Connection error	The connection attempt threw an error. See Troubleshooting
II Pending approval	A project-scoped server you haven't approved yet. See Edit .mcp.json directly

3

Use the server

Start a session and ask Claude to use the new server by name:

```
claude
```

```
Use the claude-code-docs server to look up what MCP_TIMEOUT
does
```

- ❗ You don't normally need to name a server in your prompt, since Claude chooses relevant tools on its own. Naming it here guarantees the demonstration goes through the new server rather than another tool, such as web fetch, that could answer the same question.

The first time Claude calls the server, it asks for permission to use the new tool. Approve it to continue. The tool call in Claude's output is labeled with the server name, which is how you confirm the answer came from the MCP server rather than Claude's built-in knowledge.

Remove the server

This step is optional. When you're done experimenting, you can remove the server:

```
claude mcp remove claude-code-docs
```

❗ Each connected server takes some space in [Claude's context window](#) because its tool names and server instructions load into every session. Removing servers you no longer use keeps that space free.

Where servers are saved

The `claude mcp add` command writes the server's details to a configuration file. By default it registers the server at `local` scope: private to you, active only in the current project. Pass `--scope user` to register it once for all your projects, or `--scope project` to share it with teammates.

[Change server scope](#) walks through both.

❗ `claude mcp add` works the same in every shell, including PowerShell and Command Prompt. Inside a `claude` session, use the `/mcp` command to check and manage servers you've already added.

There are other ways to add a server, each covered later on this page:

- **Add a local server**: run a program on your machine instead of connecting to a URL.
- **Edit `.mcp.json` directly**: write the JSON entry yourself instead of using the command.
- **Connect a server that requires sign-in**: add a hosted server that needs a browser sign-in before its tools work.

Find your configuration on disk

The `claude mcp add` command writes the server to one of three scopes, stored across two files, depending on the `--scope` flag. You don't need to edit these files directly, but knowing where they are helps with debugging and version control.

Scope	File	Available to
local	~/.claude.json , under the entry for this project	Only you, only this project. The default
project	.mcp.json in your project root	Everyone who clones the project
user	~/.claude.json , under the top-level mcpServers key	Only you, all projects

On Windows, `~/.claude.json` resolves to `%USERPROFILE%\ .claude.json` , typically `C:\Users\YourName\ .claude.json` . If you've set `CLAUDE_CONFIG_DIR` , Claude Code reads `.claude.json` from inside that directory instead.

Run `claude mcp get claude-code-docs` to see which scope holds a server's definition. For how the scopes interact when the same server is defined in more than one, see [MCP installation scopes](#).

Change server scope

A server's scope is fixed when you add it, so changing scope means removing the entry and re-adding it at the new one. Both cases below start by removing the local entry from the first walkthrough, so the server has only one definition. If you already removed it at the end of that walkthrough, skip this command:

```
claude mcp remove claude-code-docs --scope local
```

Use a server in all your projects

Re-add the server at `user` scope to make it active in every project you open, still private to you:

```
claude mcp add --scope user --transport http claude-code-docs
https://code.claude.com/docs/mcp
```

Share a server with your team

Re-add the server at `project` scope, which writes to `.mcp.json` in the project root:

```
claude mcp add --scope project --transport http claude-code-docs
https://code.claude.com/docs/mcp
```

Commit `.mcp.json` to version control. Teammates who clone the repository and start Claude Code see a prompt to approve the server, then it connects for them too.

Additional MCP server examples

The first walkthrough used a hosted server that connects without any sign-in. The examples below cover the other two common shapes, with the same add, check, use flow.

Add a local server

A local stdio server is a program Claude Code starts as a subprocess on your machine, rather than a service it reaches over a URL. Use one for tools that need access to local resources like a browser, your filesystem, or a database socket.

The **Playwright MCP server** is a good one to try: it gives Claude a browser it can navigate, click, and read, and it needs no account. It runs through `npx`, so it requires **Node.js** 18 or later.

1 Add the Playwright server

Register the server with the command Claude Code should run to start it:

```
claude mcp add playwright -- npx -y @playwright/mcp@latest
```

This command differs from the hosted example in three ways:

- There's no `--transport` flag, because local servers use the default `stdio` transport.
- Everything after the `--` separator is the command Claude Code runs to start the server.
- `-y` tells `npx` to install the package without prompting.

Playwright drives whichever Chrome is already installed on your machine. To use a different browser, append `--browser` with the browser name, for example `--browser firefox`, after `@playwright/mcp@latest`.

2 Check the connection

The `Added` confirmation means the entry was saved, not that the command runs. Check the connection:

```
claude mcp list
```

The first check can show `X Failed to connect` while `npx` downloads the package, so wait a moment and run it again.

3 Use the browser

Give Claude a task that needs the browser:

```
Use playwright to open https://example.com and tell me the  
page title
```

A browser window opens so you can watch it work, and the tool calls in Claude's output are labeled with the `playwright` server name and the action, like `browser_navigate`.

Try pointing it at your local dev server to check that a page still renders after a change, or have it walk through a bug report step by step.

Connect a server that requires sign-in

Hosted services like Sentry, Linear, and Notion run their MCP servers behind OAuth: you add the server's URL, then sign in through your browser.

The steps below use Sentry as the example. To connect a different service, substitute its URL, which you can find in the [Anthropic Directory](#) or the service's documentation.

1 Add the server

The `add` command is the same as for the docs server, with Sentry's URL:

```
claude mcp add --transport http sentry
https://mcp.sentry.dev/mcp
```

After adding, `claude mcp list` shows the server with `! Needs authentication`. That's expected: the next step completes the sign-in.

2 Authenticate in your browser

Start a Claude Code session and open the MCP panel:

```
/mcp
```

Select `sentry` from the list, press Enter, and choose `Authenticate`. Your browser opens to Sentry's sign-in page. Approve the connection there.

Back in Claude Code, the server's status changes to connected. If sign-in fails or the browser doesn't open, see [Troubleshooting](#).

3 Use the server

Ask Claude something that needs the service, like `What Sentry projects do I have access to?`, and look for tool calls labeled with the `sentry` server name in its output.

Servers that authenticate with a static token instead of OAuth take the token at add time with `--header "Authorization: Bearer <token>"`. See the [GitHub example](#) for a worked version.

Edit `.mcp.json` directly

Every file in the [scope table](#) uses the same JSON format for server entries. This section edits `.mcp.json`, the project-scope file. It's the one most worth writing by hand because it's checked into the repository, where it doubles as configuration-as-code for your team.

Create `.mcp.json` in your project root. The example below defines both servers from this guide, the hosted docs server reached over HTTP and the Playwright server as a local `stdio` process:


```
{
  "mcpServers": {
    "claude-code-docs": {
      "type": "http",
      "url": "https://code.claude.com/docs/mcp"
    },
    "playwright": {
      "type": "stdio",
      "command": "npx",
      "args": ["-y", "@playwright/mcp@latest"]
    }
  }
}
```

The fields differ by server type:

- For HTTP servers, `url` is the endpoint Claude Code connects to.
- For stdio servers, `command` and `args` are the program it runs.

After saving the file, start a new Claude Code session in the project. Claude Code reads `.mcp.json` at startup.

The first time Claude Code sees a project-scoped server, it asks you to approve it. The prompt exists so a repository you clone can't launch processes on your machine without your consent. Approve the prompt, or run `/mcp` to approve later if you missed it.

Once you've approved, run `/mcp` and check that the servers show as connected. If one shows an error instead, see [Troubleshooting](#).

Connect from other surfaces

This guide uses the `claude mcp` CLI commands, but every Claude Code surface can connect to MCP servers:

- **Claude Code desktop app:** add servers through the [Connectors UI](#).
- **Claude Desktop chat app:** a separate app from Claude Code. To copy servers from its `claude_desktop_config.json` into the CLI, run `claude mcp add-from-claude-desktop` on macOS or WSL.
- **VS Code:** see [Connect to external tools with MCP](#).

- **Claude Code on the web:** reads `.mcp.json` from your repository. See [Edit .mcp.json directly](#).
- **Claude.ai:** connectors you add at [claude.ai/customize/connectors](#) load automatically in the CLI when you sign in with that account. See [Use MCP servers from Claude.ai](#).

Troubleshooting

If a server doesn't connect, check its status with `/mcp` inside a session or `claude mcp list` from your shell, then match the symptom below. The `/mcp` panel also lets you reconnect or authenticate without leaving the session.

- `/mcp` shows No MCP servers configured
- Status shows Failed to connect or Connection error
- Connection timed out at startup
- Server already exists
- Server connects but no tools appear
- Changes to `.mcp.json` don't take effect
- OAuth sign-in fails or browser doesn't open

Next steps

With one server connected, explore the rest of what MCP enables:

- [Find more MCP servers](#) in the Anthropic Directory
- [Share servers with your team](#) using installation scopes
- [Manage MCP access for an organization](#) with managed settings and policy controls
- [Reference MCP resources](#) in prompts with @ mentions
- [Run MCP prompts as commands](#) from the `/` menu
- [Build your own server](#) with the MCP SDK

